Lliurament: enllaç al repositori de GitHub al Moodle. El repositori ha d'incloure el codi degudament comentat, el README.md, el .env.example i la col·lecció de Postman exportada amb les respostes desades.

📝 Enunciat: Creació d'una API REST per a un gestor de tasques

🎯 Objectiu

Crear una API REST amb Node.js, Express i MongoDB que permeti gestionar tasques. L'API ha de permetre la creació, la consulta, l'actualització i l'eliminació de tasques, així com gestionar camps específics com el cost, la previsió d'hores, les hores reals, l'estat de completada i la imatge.

📋 Requisits

- 🟢 Utilitzar Node.js com a entorn d'execució.
- ⚙️ Utilitzar Express per configurar les rutes i gestionar les peticions HTTP.
- 🗂️ Utilitzar MongoDB com a base de dades per guardar les tasques. El controlador no pot utilitzar l'array de dades simulades de la sessió.
- 🛠️ Utilitzar Mongoose per modelar les dades i gestionar la comunicació amb la base de dades.
- 🌐 L'API ha de seguir els principis de REST:
  - GET 🔍 per obtenir dades.
  - POST ➕ per crear dades noves.
  - PUT/PATCH ✏️ per actualitzar dades.
  - DELETE 🗑️ per eliminar dades.

⚡ Funcionalitats principals

L'API ha de permetre:

1. ➕ Crear una nova tasca

- Especificar:
  - 🖊️ Títol: nom de la tasca.
  - 📄 Descripció: detalls de la tasca.
  - 💰 Cost: cost estimat de la tasca.
  - ⏳ Previsió d'hores: temps estimat per completar la tasca.
  - ⌛ Hores reals: temps real invertit en la tasca.
  - 🖼️ Imatge: URL o ruta d'una imatge relacionada amb la tasca.
  - ✅ Estat: indicar si la tasca està completada o no.
- 🕒 La data de creació s'ha de generar automàticament.

- 📅 Quan una tasca es marca com a completada (en crear-la o en actualitzar-la), s'ha de guardar automàticament la data de finalització (finished_at). Si una tasca completada es torna a marcar com a no completada, finished_at ha de tornar a ser null.

2. 🔍 Obtenir totes les tasques

- Retornar una llista amb totes les tasques disponibles.

3. 🔎 Obtenir una tasca específica

- Consultar una tasca pel seu ID.

4. ✏️ Actualitzar una tasca

- Modificar qualsevol camp d'una tasca:
  - 💰 Actualitzar el cost.
  - ⌛ Actualitzar les hores reals.
  - ✅ Marcar la tasca com a completada (es guarda automàticament la data de finalització).
  - 🖼️ Actualitzar la imatge.

5. 🗑️ Eliminar una tasca

- Eliminar una tasca pel seu ID.

📂 Estructura del projecte

1. 📁 Fitxers i carpetes principals

- `models/`: conté els models de dades (`Task.js`).
- `controllers/`: conté les funcions que gestionen les operacions de l'API (`taskController.js`).
- `routes/`: conté les rutes de l'API (`taskRoutes.js`).
- `.env`: variables d'entorn (port i connexió a MongoDB).
- `index.js`: punt d'entrada; connecta amb MongoDB i inicia el servidor.
- `app.js`: configuració de l'aplicació Express (middleware i rutes).

2. 🛠️ Descripció del model Task

El model Task representarà cada tasca del sistema. Defineix els camps següents:

- 🖊️ Títol (`title`): camp obligatori que descriu el nom de la tasca.
- 📄 Descripció (`description`): camp opcional per detallar la tasca.
- 💰 Cost (`cost`): camp obligatori que indica el cost estimat de la tasca. No pot ser negatiu.
- ⏳ Previsió d'hores (`hours_estimated`): camp obligatori que especifica el temps estimat per completar la tasca. No pot ser negatiu.
- ⌛ Hores reals (`hours_real`): camp opcional per registrar el temps real invertit en la tasca. No pot ser negatiu.

- 🖼️ Imatge (`image`): camp opcional per guardar la URL o la ruta d'una imatge relacionada amb la tasca.
- ✅ Completada (`completed`): camp booleà que indica si la tasca està completada. Per defecte, el valor és `false`.
- 📅 Data de finalització (`finished_at`): camp opcional que guarda la data i l'hora en què la tasca es completa.
- 🕒 Data de creació (`createdAt`): camp automàtic que registra la data i l'hora en què es crea la tasca.

📡 Rutes de l'API

Crea el fitxer `routes/taskRoutes.js` amb les rutes següents:

Mètode	Ruta	Descripció
POST	`/api/tasks`	➕ Crear una nova tasca amb els camps especificats.
GET	`/api/tasks`	🔍 Obtenir totes les tasques disponibles.
GET	`/api/tasks/:id`	🔎 Obtenir una tasca específica pel seu ID.
PUT	`/api/tasks/:id`	✏️ Actualitzar una tasca específica pel seu ID (modificar qualsevol camp).
DELETE	`/api/tasks/:id`	🗑️ Eliminar una tasca específica pel seu ID.

ℹ️ A les URL, substitueix `:id` per l'`_id` que genera MongoDB (el trobaràs a la resposta del POST o del GET). Exemple: `http://localhost:3000/api/tasks/6734a1f2c9e7b3d1a0f4e812`

🎮 Exemples de peticions amb Postman

1. ➕ Crear una tasca

- Mètode: POST
- URL: http://localhost:3000/api/tasks
- Body (raw → JSON):

```
{
  "title": "Dissenyar una pàgina web",
  "description": "Crear una pàgina web responsiva amb HTML, CSS i JavaScript",
  "cost": 500,
  "hours_estimated": 20,
  "image": "https://example.com/images/task1.jpg"
}
```

2. 🔍 Obtenir totes les tasques

- Mètode: GET
- URL: http://localhost:3000/api/tasks

3. 🔎 Obtenir una tasca específica

- Mètode: GET
- URL: http://localhost:3000/api/tasks/:id

4. ✏️ Actualitzar una tasca

- Mètode: PUT
- URL: http://localhost:3000/api/tasks/:id
- Body (raw → JSON):

```
{
  "completed": true,
  "hours_real": 15,
  "image": "https://example.com/images/updated_task.jpg"
}
```

5. 🗑️ Eliminar una tasca

- Mètode: DELETE
- URL: http://localhost:3000/api/tasks/:id

⚠️ En les peticions GET i DELETE, el Body de Postman ha d'estar a none.

💡 Pistes

- Per obtenir una tasca pel seu ID, consulta el mètode `findById()` de Mongoose.
- La data de finalització s'ha de gestionar al controlador: comprova el valor de `completed` que arriba a `req.body` abans de guardar.
- Per a la data de creació pots utilitzar `default: Date.now` (com al model Producte) o l'opció `{ timestamps: true }` de l'esquema.

🎉 Entrega final

Quan acabis el projecte:

- Implementa el model Task seguint la descripció proporcionada.
- Comprova que totes les rutes funcionen correctament amb Postman.
- Opcional: afegeix validacions avançades o funcionalitats extra, com la paginació o els filtres.

📦 Què s'ha de lliurar

Al Moodle només s'ha de lliurar l'**enllaç al repositori de GitHub**. El repositori ha de ser públic (o compartit amb la professora) i ha de contenir:

1. El codi del projecte degudament comentat (`index.js`, `app.js`, `models/Task.js`, `controllers/taskController.js`, `routes/taskRoutes.js`).
2. Un fitxer `.gitignore` que exclogui `node_modules/` i `.env`.
3. Un fitxer `.env.example` amb les variables necessàries, sense dades reals:

```
PORT=3000
MONGO_URI=mongodb://127.0.0.1:27017/taskmanager
```

4. Un fitxer `README.md` amb les instruccions per instal·lar i arrencar el projecte (`npm install`, crear el `.env`, `npm run dev`).
5. La col·lecció de Postman exportada a `postman/gestor-tasques.postman_collection.json`.
6. Un mínim de 4 commits amb missatges que expliquin el que s'ha fet a cada pas.

📮 Col·lecció de Postman

La col·lecció ha de contenir aquestes 7 peticions, i cada una ha de tenir la resposta desada com a Example:

1. POST: crear una tasca correcta (resposta 201).
2. POST: crear una tasca sense title o sense cost (resposta 400).
3. GET: obtenir totes les tasques.
4. GET: obtenir una tasca pel seu ID.
5. GET: obtenir una tasca amb un ID que no existeix (resposta 404).
6. PUT: marcar una tasca com a completada (s'ha de veure el camp finished_at amb la data).
7. DELETE: eliminar una tasca.

Com fer-ho:

- Desar la resposta: després de fer Send, a la part de la resposta, Save Response → Save as example.
- Exportar la col·lecció: als tres puntets de la col·lecció, Export → Collection v2.1, i guardar el fitxer a la carpeta postman/ del projecte.